REPORT TECNICO

BUILD WEEK – PROGETTO DI RETE PER LA COMPAGNIA THETA

Sborotech

SOLUTIONS & INNOVATION

INTRODUZIONE

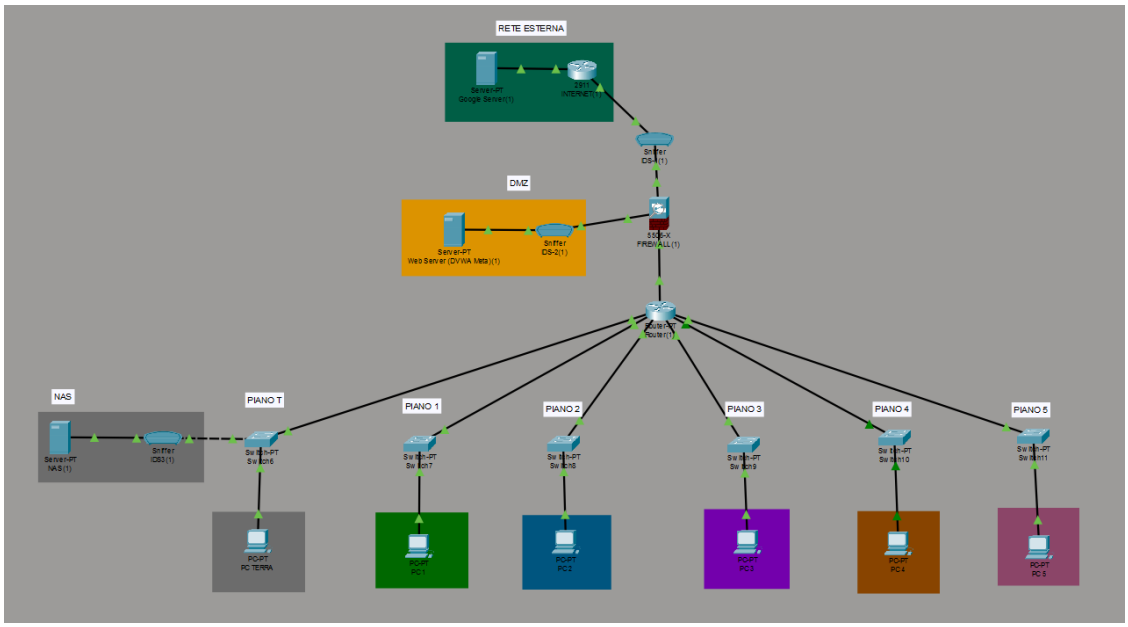
Questo report sarà redatto in una maniera **meno formale**, per **spiegare** i lati tecnici del progetto e **illustrare** i procedimenti tecnici del funzionamento della rete e degli script in Python.

TOPOGRAFIA

HARDWARE FORNITO DALL'AZIENDA:

A. DISPOSITIVI PREVISTI:
   - 20 computer per piano, per un totale di 120 computer;

B. COMPONENTI AGGIUNTIVI:
   1. 1 Web server
   2. 1 Firewall perimetrale
   3. 1 NAS (Network Attached Storage)
   4. 3 IDS/IPS (Intrusion Detection System / Intrusion Prevention System)



PROCEDIMENTO

La configurazione di rete che ci è stata commissionata prevede una struttura di 6 piani, con 20 computer collegati ad uno switch per piano:

- In primis, lavorando sulla topologia di rete su **Cisco Packet Tracer**, abbiamo disposto su ogni piano dell'edificio **20 PC** collegati ad uno switch, come richiesto.

- Successivamente abbiamo collegato i 6 switch ad un **router centrale**, posizionato al piano terra, andando a dividere ogni piano in **subnet diverse** e abilitando dal router la funzione server **DHCP**, di modo che tutti e 120 i dispositivi ricevessero un indirizzo IP unico dal router, appartenente alla subnet del piano in cui sono stati posizionati.

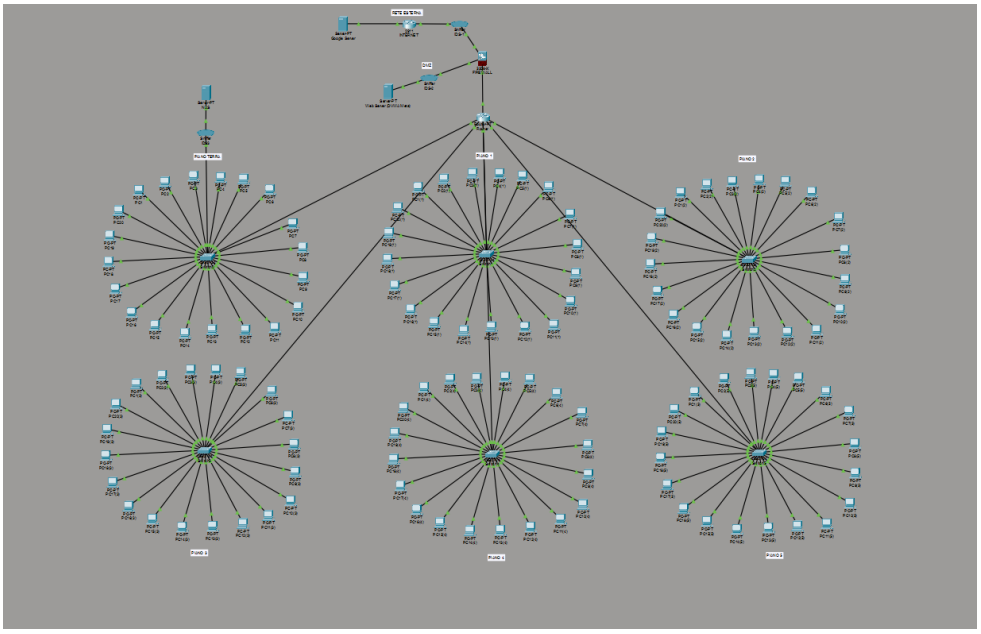
- In aggiunta, sempre al piano terra, abbiamo posizionato il **NAS**, collegandolo allo **switch** tramite uno **Sniffer** (IDS-1), che analizzi tutto il traffico in entrata e specialmente quello sospetto o

malevolo, e abbiamo creato una **DMZ** (Demilitarized Zone) con all'interno il **Web server** dell'azienda collegato a un altro **Sniffer** (IDS-2).

- A quel punto abbiamo inserito il **Firewall perimetrale** (ASA 5506-X) nella configurazione, come richiesto, collegandolo al **router centrale** e allo **Sniffer** della DMZ, posizionandolo al piano terra, di modo che andasse a creare un perimetro sicuro che contenesse tutta la struttura della rete interna.

- Per quanto riguarda la configurazione del **Firewall**, tramite la **CLI** abbiamo assegnato ad ognuna delle 3 porte di rete ad esso collegate un **security level** differente, **100** (massimo) per la rete interna (inside), **50** (intermedio) per la DMZ e **0** (minimo) per la rete esterna (Internet), che in questo caso abbiamo simulato con un **router** (attraverso il terzo **Sniffer** IDS-3) a cui abbiamo collegato un server generico. In questo modo abbiamo isolato le 3 zone in base al loro livello di sicurezza.

- Abbiamo quindi aggiunto, sempre tramite **CLI**, alcune **regole** ed **eccezioni** al **Firewall**, come il **mascheramento** dell'IP privato del Web server assegnandogli **un IP pubblico** attraverso il quale possa essere raggiungibile dalla **rete esterna** (192.168.5.10 → 200.0.0.10) e il **NAT** per tutti i dispositivi della rete interna, di modo che fossero abilitati a comunicare con Internet.



RIEPILOGO E CONCLUSIONE DELLA CONFIGURAZIONE:

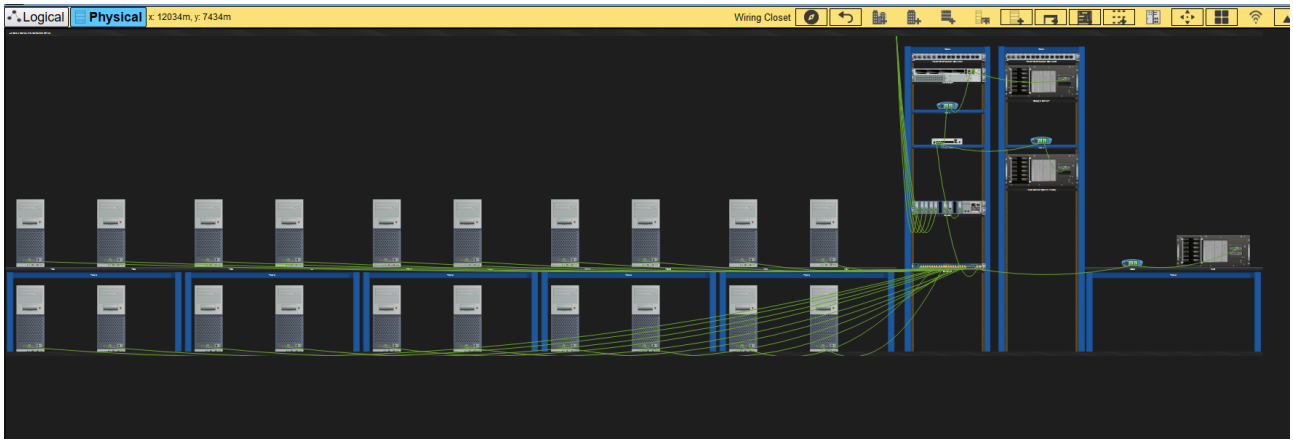
La topologia che abbiamo ideato e creato, nel modo illustrato nel precedente paragrafo seguendo le richieste dell'azienda, è **sicura**, è **funzionante** e ha le seguenti caratteristiche principali:

I. **RETE INTERNA**: ogni PC di ogni piano è in grado di comunicare con qualsiasi altro PC, di accedere al NAS e di accedere ad internet da web browser;

II. **DMZ**: il Web server non è in grado di comunicare direttamente con i dispositivi della rete interna ma può essere raggiunto da essi tramite il suo IP privato e dall'esterno tramite il suo IP pubblico o mascherato (differenze di security level)

III. **RETE ESTERNA**: l'accesso alla rete interna è bloccato dal Firewall ma può accedere alla pagina web del Web server nella DMZ.

IV. **IDS/IPS**: posizionati nei punti strategici precedentemente esplicati, permettono di monitorare tutto il traffico sospetto, malevolo o non autorizzato in entrata in primis verso il Firewall stesso (IDS-3), poi verso la DMZ con il Web server (IDS-2) e infine verso il NAS (IDS-1). Tutti e 3 gli sniffer agiscono scansionando tutte le richieste che li attraversano, individuando quelle sospette, e registrandole per successiva revisione, implementazione o manutenzione.



TEST VERBI HTTP

Come servizio a valore aggiunto, Sborotech Srl offre ai propri clienti lo sviluppo di tool personalizzati finalizzati al miglioramento della sicurezza delle infrastrutture IT aziendali, tra questi rientra un programma scritto in Python ("httptester.py") e volto alla verifica dei verbi HTTP abilitati o non abilitati su un dato URL/Web Server.

In questo caso, per testare il funzionamento del programma in uno scenario che andasse a simulare la configurazione di rete che ci è stata commissionata dalla Theta spa, abbiamo creato un laboratorio virtuale con 3 macchine:

1. **KALI LINUX** (IP: 192.168.3.100/24) = per simulare un qualsiasi PC presente all'interno della rete;
2. **METASPLOITABLE** 2.0 (IP: 192.168.4.100/24) = per simulare il web server presente all'interno della DMZ dell'azienda.
3. **PFSENSE** = simulatore di Firewall, a cui sono collegate le due macchine precedentemente elencate, appartenenti a reti diverse.
   SCHEDA 1 (WAN): 192.168.1.212/24.
   SCHEDA 2 (Lan KALI): 192.168.3.1/24.
   SCHEDA 3 (Lan META): 192.168.4.1/24.

Se si volesse visionare il codice, il file completo si trova nella cartella "Allegati",

PROCEDIMENTO DI VERIFICA SVOLTO SULL'URL DEL WEB SERVER:

Per prima cosa abbiamo inserito nel programma **l'URL** di destinazione, ovvero quello della pagina del Web Server simulato, in questo caso http://192.168.4.100/dvwa/login.php/.

Successivamente, all'avvio del programma da Kali, viene presentato all'utente un semplice menu interattivo che gli illustra quali sono i verbi HTTP per i quali può effettuare una richiesta e una verifica. C'è l'opzione di selezionarne uno, più di uno o tutti insieme (di default). Noi siamo ricorsi a quest'ultima opzione.

Fatto ciò, il programma invia al server una richiesta per ogni verbo HTTP selezionato, specificando per ognuno di essi lo status (200 = abilitato, 405 = non abilitato, 404 = non trovato ecc) e il time lapse della richiesta in ms.

PORT SCANNER

Nel presente progetto è stato sviluppato uno **scanner TCP** in Python basato sulla tecnica di **TCP connect scan**, che prevede il completamento del **three-way handshake** per determinare lo stato delle porte di un sistema target.

OBIETTIVO DEL PROGRAMMA

L'obiettivo è la realizzazione di uno strumento in grado di:

- analizzare un intervallo specificato di porte su un host target
- identificare le porte aperte
- associare, ove possibile, le porte ai servizi noti

TECNOLOGIE UTILIZZATE

La libreria principale impiegata è SOCKET, che fornisce un'interfaccia di rete a basso livello per la gestione delle comunicazioni tramite socket.

Nello specifico:

- **AF_INET**: utilizzato per indirizzi IPv4
- **SOCK_STREAM**: utilizzato per implementare comunicazioni TCP

FUNZIONAMENTO DEL PROGRAMMA

INPUT UTENTE

All'avvio del programma viene richiesto all'utente di inserire:

- **l'indirizzo IP** del target;
- un **intervallo** di porte nel formato **min-max.**

PARSING DEL RANGE DI PORTE

Il programma suddivide la stringa inserita utilizzando il carattere "-" come **delimitatore**, **convertendo** successivamente i valori ottenuti in interi per definire gli estremi dell'intervallo di scansione.

FASE DI SCANSIONE

La scansione viene effettuata iterando su ciascuna porta del range specificato. Per ogni porta:

- viene creato un socket TCP;
- viene impostato un timeout pari a 0.5 secondi;
- viene tentata una connessione verso il target.

Il metodo **connect_ex()** restituisce un codice numerico:

- **valore 0**: connessione riuscita → porta aperta
- **valore diverso da 0**: connessione fallita → porta chiusa o filtrata

Questo approccio implica l'effettiva apertura di una connessione TCP completa (three-way handshake), rendendo la scansione affidabile ma facilmente rilevabile da sistemi di sicurezza come firewall e IDS/IPS.

IDENTIFICAZIONE DEL SERVIZIO

Nel caso in cui una porta risulti aperta, il programma tenta di associare un servizio noto tramite la funzione **getservbyport()**. In caso di errore, la porta viene classificata come “Unknown”.

OUTPUT

Durante l’esecuzione viene mostrato lo stato di avanzamento della scansione. Al termine, vengono elencate tutte le porte aperte con il relativo servizio associato.

TEST E RISULTATI

Per verificare il corretto funzionamento dello strumento sono stati eseguiti diversi test. Il caso più significativo ha previsto la scansione di una macchina virtuale Metasploitable, progettata appositamente per esercitazioni di sicurezza.

TARGET: 10.20.21.2

RANGE DI PORTE: 0–3306

```
Please, insert the target IP to scan: 10.20.21.2
Please, insert the port range to scan [0-65535]: 0-3306
Scanning port 3306 (3307/3307)...
--- Scan Results ---
[✔] OPEN  -> 10.20.21.2:21 (ftp)
[✔] OPEN  -> 10.20.21.2:22 (ssh)
[✔] OPEN  -> 10.20.21.2:23 (telnet)
[✔] OPEN  -> 10.20.21.2:25 (smtp)
[✔] OPEN  -> 10.20.21.2:53 (domain)
[✔] OPEN  -> 10.20.21.2:80 (http)
[✔] OPEN  -> 10.20.21.2:111 (sunrpc)
[✔] OPEN  -> 10.20.21.2:139 (netbios-ssn)
[✔] OPEN  -> 10.20.21.2:445 (microsoft-ds)
[✔] OPEN  -> 10.20.21.2:512 (exec)
[✔] OPEN  -> 10.20.21.2:513 (login)
[✔] OPEN  -> 10.20.21.2:514 (shell)
[✔] OPEN  -> 10.20.21.2:1099 (rmiregistry)
[✔] OPEN  -> 10.20.21.2:1524 (ingreslock)
[✔] OPEN  -> 10.20.21.2:2049 (nfs)
[✔] OPEN  -> 10.20.21.2:2121 (iprop)
[✔] OPEN  -> 10.20.21.2:3306 (mysql)

====================
   SCAN COMPLETED
====================
```

La scansione è stata completata in circa **10** secondi. Sono state rilevate diverse **porte aperte**, tra cui servizi tipicamente vulnerabili come FTP (21), Telnet (23) e HTTP (80), coerentemente con la natura intenzionalmente insicura del sistema analizzato.

LIMITI DEL PROGRAMMA

L'implementazione attuale presenta alcune limitazioni:

- **Prestazioni**: la scansione è sequenziale (una porta alla volta), con conseguenti tempi elevati su range ampi
- **Rilevabilità**: la TCP connect scan completa il three-way handshake, rendendo l'attività facilmente tracciabile nei log di sistema
- **Timeout fisso**: può causare falsi negativi in condizioni di rete lente
- **Assenza di IPv6**: supporta esclusivamente indirizzi IPv4
- **Gestione errori limitata**: non distingue in modo dettagliato tra porte chiuse e filtrate

ASPETTI ETICI E LEGALI

Il port scanning deve essere eseguito **esclusivamente** su sistemi per i quali si dispone di **autorizzazione esplicita**. L'utilizzo **non autorizzato** di tali tecniche può essere considerato **attività ostile** e, in alcuni casi, **illegale**.

PACKET SNIFFER

Nel progetto di rete della **Sborotech Srl** è stato sviluppato uno script Python di packet sniffing utilizzando Scapy per monitorare il traffico nella rete interna aziendale. Il programma intercetta pacchetti TCP e UDP e analizza i principali flag del protocollo TCP, tra cui **SYN, SYN-ACK, ACK, PSH-ACK e FIN-ACK**.

```python
from scapy.all import sniff, IP, TCP, UDP

def get_flags(tcp):
    flags = tcp.flags

    if flags == 0x02:
        return "SYN"
    elif flags == 0x12:
        return "SYN-ACK"
    elif flags == 0x10:
        return "ACK"
    elif flags == 0x18:
        return "PSH-ACK"
    elif flags == 0x11:
        return "FIN-ACK"
    else:
        return str(flags)

def process_packet(packet):
    if IP in packet:
        ip = packet[IP]
        # TCP
        if TCP in packet:
            tcp = packet[TCP]
            flag = get_flags(tcp)

            print(f"[TCP {flag}] {ip.src}:{tcp.sport} → {ip.dst}:{tcp.dport}")

        # UDP
        elif UDP in packet:
            udp = packet[UDP]

            print(f"[UDP] {ip.src}:{udp.sport} → {ip.dst}:{udp.dport}")

if __name__ == "__main__":
    print("Sniffing TCP + UDP ... (CTRL+C per fermare)")
    sniff(filter="tcp or udp", store=False, prn=process_packet)
```

In particolare, lo script permette di osservare tutte le fasi della comunicazione **TCP**: APERTURA della connessione (SYN), RISPOSTA del server (SYN-ACK), CONFERMA (ACK), scambio dati (PSH-ACK) e CHIUSURA della connessione (FIN-ACK). Inoltre, per i pacchetti **UDP** vengono visualizzate le informazioni principali di trasmissione.

Durante l'esecuzione del test è stata aperta la pagina web della macchina **Metasploitable** all'indirizzo **192.168.20.10**. Lo script ha intercettato e "agganciato" tutto lo scambio di pacchetti tra client e server, permettendo di osservare in tempo reale la comunicazione HTTP generata dal browser, inclusa la fase di richiesta e risposta.

Inoltre, vengono mostrati gli indirizzi IP e le porte coinvolte, consentendo di monitorare in tempo reale il traffico tra i dispositivi della LAN. Questo strumento è utile per verificare il corretto funzionamento della rete, supportare i sistemi di sicurezza (IDS/IPS) e individuare eventuali anomalie.

```
┌──(kali㉿kali)-[~]
└─$ sudo python3 sniffer.py
Sniffing TCP + UDP ... (CTRL+C per fermare)
[TCP SYN] 192.168.10.10:55694 → 192.168.20.10:80
[TCP SYN-ACK] 192.168.20.10:80 → 192.168.10.10:55694
[TCP ACK] 192.168.10.10:55694 → 192.168.20.10:80
[TCP PSH-ACK] 192.168.10.10:55694 → 192.168.20.10:80
[TCP ACK] 192.168.20.10:80 → 192.168.10.10:55694
[TCP PSH-ACK] 192.168.20.10:80 → 192.168.10.10:55694
[TCP ACK] 192.168.10.10:55694 → 192.168.20.10:80
[TCP ACK] 192.168.10.10:55694 → 192.168.20.10:80
[TCP ACK] 192.168.20.10:80 → 192.168.10.10:55694
[TCP FIN-ACK] 192.168.10.10:55694 → 192.168.20.10:80
[TCP FIN-ACK] 192.168.20.10:80 → 192.168.10.10:55694
[TCP ACK] 192.168.10.10:55694 → 192.168.20.10:80
```

In conclusione, lo script rappresenta un efficace strumento di analisi e controllo per l'infrastruttura di rete della **Sborotech Srl**, contribuendo alla sua sicurezza e affidabilità.

CONCLUSIONE

Il progetto ha permesso di comprendere i principi fondamentali del Networking, e di approfondire e assimilare i concetti imparati in queste 3 settimane, trovandogli applicazione pratica in uno scenario di vita reale.